

CODE :

```
!sudo apt update

!sudo apt install -y build-essential cmake pkg-config

!sudo apt install -y libopencv-dev

%%writefile detect.cpp

#include <opencv2/opencv.hpp>

#include <iostream>

using namespace cv;

using namespace std;


int main() {

    // Load image

    Mat img = imread("image.jpg");

    if (img.empty()) {

        cout << "Image not found!" << endl;

        return -1;

    }


    // Convert to gray

    Mat gray;

    cvtColor(img, gray, COLOR_BGR2GRAY);


    // Edge detection

    Mat edges;

    Canny(gray, edges, 100, 200);


    // ----- Extract Lines using Hough Transform -----

    vector<Vec2f> lines;

    HoughLines(edges, lines, 1, CV_PI / 180, 120);
```

```

Mat lineImg = img.clone();
for (size_t i = 0; i < lines.size(); i++) {
    float rho = lines[i][0], theta = lines[i][1];
    double a = cos(theta), b = sin(theta);
    double x0 = a * rho, y0 = b * rho;
    Point p1(cvRound(x0 + 1000 * (-b)), cvRound(y0 + 1000 * (a)));
    Point p2(cvRound(x0 - 1000 * (-b)), cvRound(y0 - 1000 * (a)));
    line(lineImg, p1, p2, Scalar(0, 0, 255), 2);
}

// ----- Extract Contours -----
vector<vector<Point>> contours;
vector<Vec4i> hierarchy;
findContours(edges, contours, hierarchy, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);

Mat contourImg = img.clone();
drawContours(contourImg, contours, -1, Scalar(0, 255, 0), 2);

// ----- Connected Components -----
Mat bw;
threshold(gray, bw, 100, 255, THRESH_BINARY);

Mat labels, stats, centroids;
int nLabels = connectedComponentsWithStats(bw, labels, stats, centroids);

Mat compImg = Mat::zeros(img.size(), CV_8UC3);
RNG rng(12345);

for (int i = 1; i < nLabels; i++) {
    Scalar color(rand() % 255, rand() % 255, rand() % 255);

```

```

for (int y = 0; y < labels.rows; y++) {
    for (int x = 0; x < labels.cols; x++) {
        if (labels.at<int>(y, x) == i) {
            complmg.at<Vec3b>(y, x) = Vec3b(color[0], color[1], color[2]);
        }
    }
}
}
}

```

```

// Save results

imwrite("edges.jpg", edges);
imwrite("lines.jpg", lineImg);
imwrite("contours.jpg", contourImg);
imwrite("components.jpg", complmg);

cout << "Processing done. Images saved." << endl;
return 0;
}

```

```

from google.colab import files
files.upload()

!g++ detect.cpp -o detect `pkg-config --cflags --libs opencv4`
!./detect

```

```

from IPython.display import Image, display

```

```

print("Edges")
display(Image(filename="edges.jpg"))

```

```

print("Lines")
display(Image(filename="lines.jpg"))

```

```

print("Contours")

```

```
display(Image(filename="contours.jpg"))
```

```
print("Connected Components")
```

```
display(Image(filename="components.jpg"))
```

Output:

